Java 学习计划 · Day 5

方法、参数传递与返回值

> 今日目标：把重复代码封装成方法；理解形参、实参、返回值和 void；编写可复用的性能指标计算方法。

一、学习安排（建议 2.5～3 小时）

模块	时间	内容
方法基础与结构	30分钟	声明、调用、参数、返回值
参数与返回值	35分钟	形参/实参、void、return
方法重用与边界	30分钟	拆分职责、参数校验
性能统计实战	45分钟	封装平均值、最大值、最小值、百分位
课后练习	30～45分钟	完成4道题并提交源码和输出

二、什么是方法

方法（method）是把一段有明确职责的代码封装起来，并通过方法名重复调用。好处是减少重复、让代码更清晰，也方便单独测试。

1. 方法的基本结构

```
public static returnType methodName(parameters) {
    // statements
    return value;
}
```

例如，定义一个计算两个整数之和的方法：

```
public static int add(int a, int b) {
    int result = a + b;
    return result;
}

// Call the method
int total = add(10, 20);
System.out.println(total); // 30
```

2. 逐项理解

public：访问修饰符；static：静态方法，可通过类名或当前类直接调用；int：返回值类型；add：方法名；a、b：形参；调用时传入的10、20：实参；return：把结果返回给调用者。

三、参数、返回值与 void

1. 形参与实参

形参是方法定义时的参数变量；实参是调用方法时实际传入的值。

```
public static double average(int total, int count) {
    return total * 1.0 / count;
}

double avg = average(1610, 7);
// total and count are formal parameters
// 1610 and 7 are actual arguments
```

2. 返回值

方法声明的返回值类型必须与 return 返回的数据类型兼容。int 方法返回整数，double 方法返回小数，String 方法返回字符串。

3. void

void 表示方法不返回结果，常用于打印、展示或执行动作。

```
public static void printResult(double average) {
    System.out.println("Average: " + average);
}

printResult(230.0);
```

> 记忆：需要把计算结果交给调用者继续使用，就设计有返回值的方法；只执行动作、不需要返回计算结果时，可以使用 void。

四、把性能统计拆成方法

把 Day 4 的统计任务拆成独立方法：每个方法只负责一件事。

```
public class PerformanceStats {

    public static double average(int[] times) {
        if (times == null || times.length == 0) {
            throw new IllegalArgumentException("Invalid data");
        }
        int sum = 0;
        for (int time : times) {
            sum += time;
        }
        return sum * 1.0 / times.length;
    }

    public static int max(int[] times) {
        if (times == null || times.length == 0) {
            throw new IllegalArgumentException("Invalid data");
        }
        int result = times[0];
        for (int time : times) {
            if (time > result) result = time;
        }
        return result;
    }

    public static int min(int[] times) {
        if (times == null || times.length == 0) {
            throw new IllegalArgumentException("Invalid data");
        }
        int result = times[0];
        for (int time : times) {
            if (time < result) result = time;
        }
        return result;
    }

    public static void main(String[] args) {
        int[] times = {350, 120, 280, 90, 500, 210, 160};
        System.out.println("Average: " + average(times));
        System.out.println("Max: " + max(times));
        System.out.println("Min: " + min(times));
    }
}
```

参考输出：Average 230.0；Max 500；Min 90。可把 Day 4 的 percentileNearestRank 方法也作为独立方法加入。

五、常见错误

1. 返回值类型不匹配：声明 int 却 return 一个 String。
2. 有返回值的方法漏写 return。
3. 把打印当成返回：System.out.println(avg) 只是输出，不等于 return avg。
4. 调用时传入的实参数量或类型不符合方法定义。
5. 除数为0：计算平均值前必须检查样本数。

六、课后练习

练习1：编写求和方法

编写 sum(int[] numbers) 方法，接收一个整数数组，返回数组元素总和。用 main 调用并打印结果。

练习2：封装性能指标

分别编写 average(int[] times)、max(int[] times)、min(int[] times)
三个方法，返回平均值、最大值、最小值。空数组或 null 输入需要合理处理。

练习3：封装百分位

把 Day 4 的 nearest-rank 百分位算法封装为 percentileNearestRank(int[] times, int percentile)。方法内部先复制并排序数组，不能改变调用者的原数组；校验 null、空数组和百分位范围1～100。

练习4：组合分析

编写一个 void 方法 printReport(int[] times)，在方法内部调用 average、max、min、percentileNearestRank，打印平均值、最大值、最小值、P95。使用数据 {350，120，280，90，500，210，160}。

> 验收标准：方法能正确声明和调用；参数、返回值使用合理；计算方法不依赖 main 中的局部变量；输入校验有效；printReport 能组合调用其他方法并输出报告。提交 .java 源码与控制台结果。

今日速记：方法 = 可复用代码块；形参在定义时声明，实参在调用时传入；return 返回结果并结束方法；void 表示不返回值。